The server is not a structural requirement: identifying accidental architectural roles under journaled programs

Alvaro Rivera

2026-05-16

TL;DR

The role we call *server* — the entity that owns coordination, accepts authoritative writes, and mediates between clients — is treated across the architecture literature as a structural component of any non-trivial distributed system. Prior work has questioned the server's privileges (local-first, federation), eliminated it as a design goal (Hypercore, Holochain), or replaced it with a different topology (peer-to-peer, blockchain). None of these positions removes the server-role as a *prerequisite for software construction itself.*

This paper names the property that distinguishes the server-role from structural requirements of distributed software: it is an *accidental category.* Under a class of systems in which nodes replicate programs rather than data or state, the server-role does not survive as a necessity. The bootstrap of a new node, the last role that appeared to require a running service, can be performed by an analog artifact: a printed QR code transferred by any means and authorized by a human decision.

The construct is recognition, not prescription. It enables auditing distributed systems for whether the server-role exists in them as a structural requirement or as a contingent artifact of the persistence model. An instantiation — a port of the Ordering bounded context of `dotnet/eShop` to a journaled-program substrate, replicated across three nodes that bootstrap through paper QR codes — confirms that the predicate dissolves under the conditions.

The contribution is conceptual; the instantiation is the existence proof, not the substance.

Formal statement

Definition. A *server-role* is the architectural function of accepting authoritative writes, owning coordination, mediating between clients, and providing the

address against which other parties direct their traffic in a distributed system. The role is independent of any specific machine, deployment topology, or cloud provider; it is the function, not its incarnation.

A server-role is an *accidental category* in a class of distributed systems S when both of two conditions hold:

- **Contingency.** The role's necessity in S is traceable to a specific representational choice — what the nodes of S replicate in order to share state.
- **Dissolution.** The role loses justification when that representational choice is replaced; nothing else about the problem domain, workload, or operational context needs to change for the role to become unnecessary.

Claim. Under a class of systems in which nodes replicate programs rather than data, state, or operations, both conditions hold for the server-role. The conditions are stated in §3; the substrate satisfying them is exhibited in §4.

Evidence. Three operating nodes derived from a port of the Ordering bounded context of `dotnet/eShop` to a journaled-program substrate, bootstrapped by paper QR codes and authorized by human decisions, execute the same domain behavior as the original microservice across its operational lifecycle, without a node fulfilling the server-role at any point in time. The original deployment topology required at least one such role; the ported topology requires none.

Claims this paper makes

1. The server-role in distributed software is an accidental category: it exists contingently, as a consequence of choices about what nodes replicate, rather than structurally, as a property of the problem being solved (§3).
2. The property naming this distinction — *accidental category* — is operationalized by two conditions: contingency (the role's necessity is traceable to a specific representational choice) and dissolution (the role loses justification when that choice is replaced) (§3).
3. Under a class of systems in which nodes replicate programs rather than data, the conditions hold for the server-role: it disappears as a structural consequence, not as a design objective (§5).
4. The bootstrap of a new node — the operation that prior literature treats as the residual case requiring a server — admits a presentation in which the information transferred crosses by analog means and the authorization is exercised by humans, without any service running on either side (§4).
5. An instantiation of the conditions, constructed by porting the Ordering bounded context of `dotnet/eShop` to a journaled-program substrate and deploying it across three nodes bootstrapped by paper QR codes, dissolves the server-role across the operational lifecycle (§4, §5).
6. An unexpected consequence of journaled programs is that existing cloud and microservice ecosystems cease to be architectural dependencies and become reusable domain libraries within the program itself (§6).

7. The construct is diagnostic: it enables identifying which architectural categories in any given system are structural and which are contingent, independent of the choice to act on that information (§7, §8).

1. Introduction

This paper makes a design theory contribution in the sense of Hevner, March, Park, and Ram (2004). It identifies and names a structural pattern that prior literature has documented case by case without articulating as one (the construct); derives the conditions under which the pattern dissolves (the principles); and presents an instantiation in which those conditions hold and the pattern is absent (the existence proof). The genre is design science research: evidence is presented in the form of a working artifact rather than a controlled experiment. The contribution is conceptual; the instantiation confirms that the pattern is contingent rather than structurally necessary, not that any specific system is the only way to dissolve it.

Production software is built around an assumption that is rarely articulated as such: that a *server-role* exists somewhere in the system. The role is not the physical machine, nor the deployment topology, nor any particular cloud provider. It is the architectural function — accepting authoritative writes, owning coordination, mediating between clients, providing the address against which other parties direct their traffic. Across textbooks, reference architectures, and tooling defaults, the role is treated as a primitive of distributed software: present by default, named when necessary, removed only at great cost.

Prior literature has interrogated the server's *privileges* — its asymmetric power over data, identity, or availability — and proposed designs that reduce those privileges. Local-first software argues that data should belong to its user and survive without the cloud service. Hypercore, Dat, and related peer-to-peer systems replicate append-only logs without a privileged coordinator. Holochain inverts the data-centric model so that the agent's source chain and a shared distributed hash table together replace the central server. Federated systems pluralize the server-role across cooperating hosts. Each of these approaches treats the server-role as a *target of design* — something to redesign, redistribute, or replace. None of them removes the server-role as a *prerequisite for software construction*. In each case, the role's elimination is the explicit objective of the system; the system is built to satisfy that objective.

This paper takes a different position. Under a class of systems in which the substrate shared between nodes is the program — not the data, not the state, not even the operations — the server-role does not survive as a necessity. It is a category whose referent dissolves. *In the framing developed here, server ceases to be a prerequisite for making software. Bootstrap — the last role that seemed to need a service — can be performed by a printed QR code.*

The paper develops three claims in cascade. First, the server-role is identified as an *accidental category*: a role that exists contingently in distributed software, as

a consequence of choices about what nodes replicate, and that has no referent under different choices. Second, the role's disappearance is presented not as a design goal but as a structural consequence of those different choices: in the class of systems where programs are journaled and replayed against shared domain libraries, the role's elimination is what the substrate produces, not what the engineer pursues. Third, the displacement extends beyond the role itself: under the same conditions, existing cloud and microservice ecosystems cease to function as architectural dependencies and become reusable domain libraries that programs can invoke at will. The cloud is not removed; its position in the system is restated.

The instantiation used to confirm these claims — a port of the Ordering bounded context from Microsoft's `dotnet/eShop` reference application, executed across three nodes that share a journaled domain library and bootstrap each other through paper QR codes — is presented as existence proof, not as the contribution of the paper. The same construct admits other instantiations: any system that satisfies the conditions derived in §3 would dissolve the server-role in the same way. The choice of `dotnet/eShop` is rhetorical — a canonical cloud-microservices reference whose original deployment topology makes the contrast visible — rather than essential to the argument. The framework underlying the instantiation was not designed to support this paper's claim. The property is identified retrospectively, after a chain of unrelated architectural decisions converged on a system in which the server-role had become contingent. The work of this paper is, in that sense, a forensic reading of a class of artifacts in which the property happens to hold.

Section 2 situates the paper among prior work on local-first, peer-to-peer, agent-centric, and federated systems, identifying the dimension along which each work treats the server-role and the dimension along which the present construct differs. Section 3 defines the construct formally — *accidental category* — and derives the two conditions, contingency and dissolution, that distinguish it from a structural requirement. Section 4 presents the instantiation: the domain-library port, the three-node deployment, the analog bootstrap. Section 5 examines the first consequence: the dissolution of the server-role across the operational lifecycle. Section 6 examines the second consequence — the one that prior literature does not anticipate: the conversion of existing cloud ecosystems from architectural dependencies into reusable domain libraries. Section 7 examines limitations, counter-arguments, and the boundary of the construct's applicability. Section 8 concludes.

2. Related work

Local-first software (Kleppmann et al., 2019) articulates seven ideals for software that operates primarily on devices owned by their users rather than on remote services. Replication is mediated by Conflict-free Replicated Data Types (CRDTs), which allow concurrent edits to merge without coordination. The position with respect to cloud infrastructure is precise: cloud services are ad-

missible, but the software must continue to function in their absence. The title of the paper — *Local-First Software: You Own Your Data, in spite of the Cloud* — captures the stance: the cloud is neither rejected nor required. The server-role is reduced to one of several optional services, and its elimination is the explicit objective of the architectural pattern.

Peer-to-peer append-only logs constitute a second line. Secure Scuttlebutt (Tarr, 2014) replicates per-identity signed logs across a gossip network without a central coordinator; the log, not the data it carries, is the unit of synchronization. Hypercore and the Dat protocol generalize the same pattern, treating the append-only feed as a primitive replicated by a distributed hash table. In each case the unit of replication is a log of data — events, document edits, file fragments — and the application logic that interprets the log lives in the consuming application. The server's elimination is the design target of the network layer.

Holochain (Harris-Braun and Brock) makes the agent the source of truth. Each agent maintains a local source chain — an immutable, signed sequence of its own transactions — and contributes to a validating distributed hash table. The architecture inverts the data-centric assumption of blockchain rather than addressing the cloud directly; the elimination of the server-role is a consequence of the agent-centric inversion. The replicated artifacts are the agent's chain and the validated DHT; application logic, written as zomes, is interpreted locally by each agent.

Federated systems form a fourth line. Matrix federates an event graph across cooperating homeservers; ActivityPub federates streams of activities; the AT Protocol of Bluesky splits hosting across a Personal Data Server, a Relay, and an AppView, with account portability as the displacement mechanism. The server-role is preserved but pluralized: every participating host fulfills the role for some subset of users. The cloud is partitioned across cooperating providers rather than centralized or rejected.

Adjacent work admits brief mention. Solid (Berners-Lee, 2016) proposes Personal Online Datastores that decouple identity from application providers; the server-role moves from application owner to data host without being eliminated. Earthstar pursues local-first synchronization without a distributed hash table, sitting between Kleppmann's CRDT model and Hypercore's gossip topology. Neither treats the server-role as a category whose referent is contingent.

Each of these lines treats the server-role as a *design target* — an architectural feature to redesign, redistribute, replace, or pluralize. None of them names the property that the present paper identifies: that the role is contingent on a choice about what nodes replicate, and that under a different choice the role has no referent at all. Table 1 summarizes the four families and the position of the present construct.

Table 1. Five families of prior work and how each treats the server-role. The bottom row identifies the construct of the present paper; the columns describe what each family replicates between nodes, where its application logic resides,

why a privileged server is absent (or pluralized), and how each family situates itself with respect to cloud infrastructure.

Family	What is replicated	Where the logic lives	How the server-role is treated	Relation to the cloud
Local-first (Kleppmann)	CRDTs / data	In the application	Eliminated as a design goal	Survival without it
Hypercore / Dat / SSB	Append-only logs	In the application	Eliminated as a network goal	Replaced by peers
Holochain	Source chain + DHT	In the agent	Eliminated by agent-centric inversion	Treated as an alternative architecture
Federated (Matrix / ActivityPub / AT Protocol)	Events / activities	On federated servers	Pluralized across hosts	Fragmented across providers
The present construct	**Programs**	**In the journal of each node**	**Not a pre-requisite (accidental category)**	**Subsumed as a library**

The discriminator is the fourth column. The four families treat the server-role as something to be designed away — a residual feature whose elimination is the goal. The present construct identifies the role as contingent: under a representational choice in which nodes replicate programs rather than data, state, or operations, the role's necessity does not survive. The fifth column is downstream of the fourth: under the present construct the cloud is neither evaded, replaced, nor pluralized, but reused as a library within the program.

The substrate of §4 combines the actor model (Hewitt, 1973), event sourcing (Fowler, 2005), and CQRS (Young, 2010), synthesized as practical patterns by Vernon (2015). An additional move — domain classes carrying no persistence concerns, discovered by reflection — antedates the synthesis by roughly a decade in the framework underlying the instantiation; that genealogy is described in §4.0.

This paper is the seventh in a series. Papers 1–6 develop the substrate-level foundations on which the present construct rests, culminating in Paper 6's *infrastructural symptom*, of which the present construct extends the discrimination from layers of infrastructure to architectural roles.

The contribution unique to this paper is conceptual. The construct *accidental category* names a property that prior work documents piecewise — across local-first, peer-to-peer, agent-centric, and federated systems — without unifying. The two conditions, contingency and dissolution, formalize the property. The discrimination table of §3 operationalizes it without requiring adoption of any particular alternative. The instantiation of §4 demonstrates that the construct's predicate dissolves under the conditions; the demonstration is not the contribution.

3. The construct: accidental category

A server-role is an *accidental category* in a class of systems S when both of two conditions hold.

The contingency condition. The role's necessity in S is traceable to a specific representational choice — what the nodes of S replicate in order to share state. The choice must be identifiable: not *"the system uses a server because that is how distributed systems work"*, but *"the system uses a server because the nodes replicate property R, and the role compensates for the coordination that R imposes."* When the contingency condition holds, asking *what would happen if R were replaced?* yields a non-trivial answer.

The dissolution condition. When that representational choice is replaced — by adopting a substrate in which R is replaced by a different property — the role loses justification. The substitution's effect must be sufficient: nothing else about the problem domain, workload, or operational context needs to change for the role to become unnecessary.

Both conditions must hold. Contingency without dissolution describes a role whose justification survives substrate change — a structural requirement of the problem domain. Dissolution without contingency is not constructible: a role cannot lose justification under a substrate change unless its justification was tied to the substrate in the first place. **The boundary between accidental category and structural requirement is not a property of the role-as-name but of the function-for-which-it-is-used.**

The construct is not a universal accusation. Many roles in distributed systems are structural requirements — identity providers backed by cryptographic root authority, gateways mediating external systems, computationally expensive services such as analytics or inference engines. No substrate change redistributes the authority of an identity provider, removes the integration problem of an external network, or makes intrinsically expensive computation cheap.

The discrimination operates per role-in-use-case. The same node, identified in a deployment as "API server," instantiates an accidental category when deployed to accept authoritative writes against a shared database, and a structural requirement when deployed to enforce cryptographic identity claims. The discriminator is the function, not the name.

Seven categories of architectural role are encountered regularly in production deployments and admit discrimination under the two conditions. The third column describes, in substrate-property terms rather than system terms, what a substrate satisfying the two conditions offers in place of each role.

Table 2. Discriminator: seven canonical categories of architectural role under the two conditions of §3. The table identifies, for each role, the representational choice that necessitates it and the substrate property that would render it unnecessary.

Architectural role	Representational choice that necessitates it	What a substrate satisfying the conditions offers instead
Server as write authority	Nodes replicate data; consistency requires a single point of acceptance	Each node accepts writes locally; the journal records the program, which replays deterministically on every node
Master / primary in replication	Nodes replicate state diffs; a primary must order them	Each node owns its journal; cross-node causality is recorded by reference, not orchestrated
Coordinator in distributed transactions	Multiple writers contend for a logical entity across processes	Each logical entity has a single point of authority intrinsic to the substrate; serialization is not externalized
Stateful gateway	Clients cannot directly invoke deferred work without losing causal record	Deferred work is journal-recorded; the substrate carries causality, not the gateway
Application server / runtime container	The runtime, the framework, and the application require disparate substrates	A minimal substrate suffices to run the domain artifact
Bootstrap service	New nodes require an issuer of credentials and addresses	Bootstrap is information transfer that crosses by any means, including analog artifacts; no service runs

Architectural role	Representational choice that necessitates it	What a substrate satisfying the conditions offers instead
Orchestrator as coordination substrate	Stateful coordination across instances requires external choreography	Coordination is between actors; the substrate provides location and continuity intrinsically

The seven rows are not exhaustive; service discovery, queue-as-glue, and configuration management admit similar analysis (§7). The construct is diagnostic, not prescriptive: it enables auditing existing systems under the two conditions, independently of any decision to adopt the substrate of §4.

4. Instantiation

4.0 Genealogy

The substrate underlying the present instantiation is the journaled actor-native framework whose architectural lineage is documented in Paper 6 §4.0 and is not retraced here. The implementation source will be released open-source with the publication of this paper; the references in this section to specific test methods, file paths, and orchestrator scripts are pointers into the forthcoming release. Reproduction details are consolidated in Appendix A.

4.1 The substrate

The substrate provides three properties that together satisfy the two conditions of §3 for the server-role. First, each node owns a local journal of programs — DSL scripts that, on replay, reconstruct the node's state deterministically. Second, the domain library — the classes and verbs that the journal scripts invoke — is loaded by reflection at startup; no node hosts the domain as a service for others. Third, replication is journal-to-journal: nodes exchange script references and parameters, not state diffs, and reach convergence by replaying the same sequence of scripts against the same library. None of these properties requires a node to fulfill the server-role; each is intrinsic to the substrate, not externalized.

The substrate is described in the abstract throughout this section. Its concrete instantiation as the framework that this paper's existence proof builds upon is the one developed across the preceding papers of the series. The choice of that framework is rhetorical — it is the one whose source is available and whose author can speak to its properties — not essential to the construct.

4.2 The port

The instantiation ports the *Ordering* bounded context of Microsoft's `dotnet/eShop` reference application — a canonical example of microservice-oriented cloud architecture — onto the substrate of §4.1. The diagnostic value of the port is what it makes visible: the lines of the original codebase that the substrate cannot host without modification are a small minority, concentrated in persistence-and-deployment glue. The aggregate, the value objects, the lifecycle verbs, and the invariants move into the journaled substrate without modification. What was previously named "the Ordering microservice" turns out to be a clean domain library wrapped in infrastructure compensating for a different substrate; the port separates the two.

4.3 The three-node deployment

Three identical processes — each running the same substrate with the ported domain library loaded by reflection — deploy across three Docker containers. The coordination mesh is fully connected: three pairwise channels forming the complete `N(N-1)/2` graph at `N = 3`. Three is the minimum peer count at which the dissolution claim is unambiguous: with two peers, any rotation can be read as one serving the other; with three, no subset of two dominates the third.

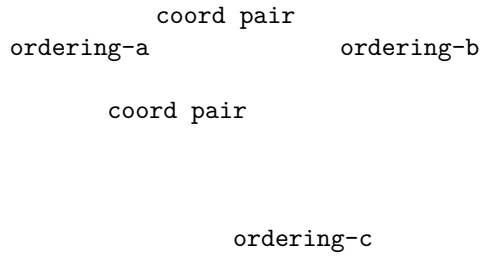

Figure 1. The three-node coordination mesh. Each node is a peer; none is privileged.

No node carries the write-acceptance privilege permanently; the role rotates symmetrically across peers. After the rotation completes, the three journals are byte-coincident.

4.4 The analog bootstrap

The bootstrap of a new peer proceeds in five phases. **F1** opens an invitation on the issuer side. **F2** transmits the invitation to the joining peer by any means, including analog ones — printed paper, a verbal dictation, an email. **F3** opens an authenticated connection back to the issuer. **F4** pauses for a human decision to approve or reject. **F5** seals a credential to the joining peer's public key and closes.

Two of the five phases are sub-software: F2 (the information transfer) and F4 (the authorization decision). Neither requires a service to be running anywhere; the carrier of F2 is whatever the operator chooses, and the decision-maker of F4 is human. The three software-mediated phases (F1, F3, F5) are local to the issuer process, which exits when the bootstrap is complete. **The issuer process does not act as a server, coordinator, authority, registry, or rendezvous point after bootstrap; its only role is to transfer a credential that allows peers to recognize each other.**

After all three containers complete the handshake — each with its own QR code carried by analog means and approved by a human — the issuer process exits. The containers continue to operate.

```
ordering-a   Up 11 seconds   0.0.0.0:6443->6443/tcp
ordering-b   Up 11 seconds   0.0.0.0:6444->6443/tcp
ordering-c   Up 11 seconds   0.0.0.0:6445->6443/tcp
```

Output of `docker compose ps` after the issuer exits. No surviving network path connects the containers to the issuer; the bootstrap papers are in a drawer.

In the framing developed here, the information that constituted the bootstrap traveled by paper; the decision that authorized it was made by a human; the cryptographic protection of the credential happened during the brief lifetime of the issuer process. The artifact that survives is the three running containers.

4.5 The empirical matrix

The demo runs across a two-by-two matrix: *in-process* versus *cross-container*, and *inline* versus *parametric*. Each cell is a complete execution of the workload across the three nodes; the in-process row reduces the bootstrap to direct invocation, while the cross-container row runs the full §4.4 protocol.

Table 3. Empirical matrix of the existence proof. Each cell reports the final journal entry count, identical on all three peers and byte-coincident on disk.

Workload regime	In-process (3 Stages, single OS process)	Cross-container (3 Dockers, real TLS + analog bootstrap)
Inline (literal scripts)	22 entries	22 entries
Parametric (Define + bind)	7 entries	7 entries

The four cells produce identical final entry counts per workload regime, and the three peers in each cell are byte-coincident on disk. The cross-container row confirms that the operational properties of the substrate survive the addition of the analog bootstrap and the real cryptographic stack; the in-process row is the structural rehearsal that isolates the substrate's intrinsic properties from

the network. The compaction from inline to parametric is real and measured; its precise ratio and the conditions under which it varies are treated in Paper 2.

In the parametric regime, the journal records seven entries rather than five: each rotation Director emits its own Define + Invocation pair, because the Define cache is local to the Stage that emits it, not replicated. The journal is the catalog of each peer's declarative vocabulary; the cache's compactness lives on the Stage axis, not on the journal axis.

4.6 Partition tolerance and re-join

The same artifact additionally exhibits partition tolerance and re-join from disk. A peer can exit the cluster while the others continue operating; a fresh process can mount the absent peer's data directory and rehydrate its journal-relative state from disk alone, without contacting the issuer or the surviving peers; and the rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up. The five-phase decomposition of this scenario and its implications for the construct's dissolution claim are described in §5. The relevant observation for §4 is that the same artifact that produces the four cells of Table 3 also produces, without modification, the operational signature of partition tolerance and re-join from disk.

5. The first consequence: dissolution of the server-role

§3 defined the server-role as a composite of four architectural functions. §4 instantiated a substrate in which each of these functions is distributed intrinsically rather than externalized to a privileged node. This section reports the dissolution as it occurs across the operational lifecycle of the cluster.

Acceptance of authoritative writes and ownership of coordination dissolve together under Director rotation. The Director role traverses the three peers in turn; no node carries the write-acceptance privilege permanently; the privilege is a transient mode that any peer can assume. The same property holds across the workload's non-happy paths — cancellation, error recovery — not only across its main sequence.

Mediation between clients and provision of the cluster's address dissolve into the peer-to-peer mesh. No node carries an authoritative address for the cluster as a whole; a request directed at any peer is acceptable and that peer can act as Director for it. The "address against which other parties direct their traffic" — the fourth limb of the §3 definition — is not a single address but the union of three pinned addresses, none of which is privileged.

The strongest empirical signature of the role's dissolution is the partition-tolerance scenario introduced in §4.6. One peer leaves the cluster; the remaining two continue to operate; the absent peer rejoins by reading its on-disk journal. Five phases describe the sequence:

- **R1.** The three peers converge to a shared journal state.

- **R2.** One peer exits; the others continue. The cluster retains quorum, retains a Director, and accepts new work.
- **R3.** The surviving Director issues additional work; the two surviving journals advance; the absent peer's on-disk journal is unchanged.
- **R4.** A fresh process — distinct in-memory identity, same data directory — reads the disk and reconstructs the journal-relative state of the absent peer, without contacting the issuer and without contacting the surviving peers.
- **R5.** The rehydrated peer integrates into the still-running cluster via peer-to-peer catch-up; the gap is replayed from a surviving peer; subsequent work brings all three peers back to a shared state.

The properties exercised by this scenario are not separable. The cluster's continued operation in the absent peer's absence is only possible if no node is the cluster's source of authority. The rehydration without consulting the issuer is only possible if the disk is a self-contained witness of the peer's prior participation. The catch-up between peers is only possible if any surviving peer is capable of replaying the gap; no specialized recovery node is invoked. *Operationally, the scenario shows that a peer can leave the cluster, the cluster can continue accepting work in its absence, and the peer can re-join by reading the journal that survived its disappearance on its own disk — no central authority is consulted at any point in this loop. The role of the issuer was discharged once, at the original onboarding, and is not invoked again.*

The issuer is invoked exactly once per peer. The credential sealed during the bootstrap of §4.4 is preserved across restarts, and the peer's continued participation is verified by the journal's cryptographic integrity, not by reference to a running service. If a peer loses its disk, it re-onboards as a new peer; if it preserves its disk, it returns as itself, regardless of how long it was absent.

The four functions of the §3 definition, taken together, are what the literature names "server." The §4 substrate dissolves them in turn: write acceptance and coordination ownership distribute across the three peers via rotation; mediation and addressing distribute across the mesh; partition tolerance and re-join distribute across the surviving peers via catch-up; the bootstrap issuer participates once and is absent thereafter. **The server is not a permanent identity. The server is a transferable coordinator role that any peer can assume in turn — and any peer can leave the cluster and re-join without reference to a coordinator outside it.**

6. The second consequence: cloud as library, not infrastructure

A second operational consequence becomes visible in §4 once the cluster continues to operate after bootstrap without any of the services that originally constituted the Ordering microservice. Cloud and microservice ecosystems, when accessed from a journaled program, are observable as libraries invoked by the

program rather than as services required for the cluster to exist.

The difference becomes operationally visible in what the cluster does when the external system is unreachable. A dependency's absence is an outage: the dependent system stops functioning because its precondition is missing. A library's absence is a failed invocation: the program records the failure and continues. The two relations differ in what the dependent system does at the moment of unavailability.

After the port described in §4.2, none of the services that previously had to be running for the Ordering microservice to exist are reachable from the cluster. The domain code runs entirely inside each node; what previously required an API host, a database, and an event bus is now a library invoked by journaled programs. The cluster operates against its own journals; the cloud topology of the original microservice has no point of contact with the cluster after the bootstrap of §4.4 completes.

The same relation generalizes to external services that the cluster invokes during normal operation. If the identity provider is unreachable at the moment of invocation, the journal records the failed invocation and the cluster continues. If a payment gateway is unreachable, the journal records the attempt and its outcome. The external service's absence is recorded as data, not experienced as outage. The cluster does not inhabit any of these services; it invokes them at chosen moments through journaled Reactions (Paper 3) and proceeds with their result, including the result of their absence.

This operational relation inverts the formulation that has organized the local-first software literature:

Table 4. The inversion of Kleppmann's formulation, captured as a six-word pair.

Kleppmann (Local-first)	The present construct
Own your data in spite of the cloud.	*Use the cloud without depending on it.*

Local-first software treats the cloud as something the software must survive without. The cluster of §4 neither survives without the cloud nor depends on it; it invokes cloud services when useful and proceeds without them when not. The two relations differ in what the absence of a cloud service means for the cluster's continued existence.

The experiment shows that cloud services, when used from a journaled program, are consumed as invocable capabilities rather than as the substrate in which the program must reside.

7. Limitations and counter-arguments

The instantiation of §4 carries specific limitations that follow from the choices made for the demo. They are reported here honestly; none of them affects the construct's predicate, but each affects what the present experiment alone establishes.

Identity in the port is local. The original `dotnet/eShop` deployment uses an external identity provider; the port substitutes a local string-typed identifier. The construct does not predict that identity providers dissolve under the conditions of §3 — identity providers backed by cryptographic root authority are structural requirements, as Table 2 notes — and the demo does not exhibit one. A second experiment that integrates an external identity provider as a Reaction-invoked library would extend the empirical surface without changing the construct's claim.

Cross-context dependencies are not portrayed. The Ordering bounded context of `dotnet/eShop` carries references resolved by the Catalog and Basket bounded contexts; the port treats those references as opaque identifiers. The construct predicts that the same port mechanism applies to Catalog and Basket; the present experiment exhibits Ordering only.

The empirical matrix is at N = 3. The mesh in §4.3 connects three peers — the minimum non-trivial fully-connected mesh. The construct's predicate is not bounded at N = 3; it dissolves the server-role whenever the conditions of §3 hold for the substrate, which is independent of node count. Scaling to larger N changes the network cost of the mesh; it does not change the conditions of §3.

The cross-container partition-tolerance scenario is captured in a parallel artefact, not in the matrix of §4. The natural cross-container equivalent of the in-process scenario in §4.6 — stopping a container, allowing the running cluster to continue accepting work in its absence, restarting the container, and watching it rehydrate from its mounted volume and rejoin via catch-up against a surviving peer — is implemented in the same orchestrator that produced the matrix (the `--rehydrate-demo` flag of the demo script) and depends on no substrate feature that is not already exercised by the in-process test in §4.6. The decision to keep §4 to the four cells of the matrix, rather than expanding to a fifth cell, was rhetorical: the in-process exercise establishes the property structurally, and §5 reasons from there. The cross-container capture documents the same property under a different process boundary; it does not add a new claim.

The compaction ratio between inline and parametric workloads depends on Director rotation. Each new Director journals a fresh Define + Invocation pair, so the Define cache compacts on the Stage axis rather than on the journal axis. The compaction is real and measured at three rounds; its precise value at larger workloads is treated in Paper 2.

A reviewer may object: *did the experiment really remove the server, or did it*

rewrite the controllers under a different name? The discriminator is operational. The server-role of §3 is measured by what happens when the candidate node is removed. After the bootstrap of §4.4, the issuer process exits; no node in the running cluster fulfills the four functions of the §3 definition; the partition-tolerance scenario of §4.6 shows that the cluster continues to operate when any single peer is removed. The role is not present in the topology to be removed; it does not have an incumbent. The objection presupposes a role that the experiment shows has no referent.

A second objection: *identity, authority, and payment are intrinsically central, so the cluster is not really server-free.* The construct does not claim otherwise. §3 separates structural requirements (identity providers, external systems, expensive computation) from accidental categories. The cluster's discharge of the server-role for the Ordering use-case does not imply that no node in any larger deployment ever fulfills a server-role; it implies that the role is not a structural property of Ordering. The same per-use-case discrimination applies to identity: when a deployment attributes credential validation to a cryptographic root authority, that authority's role is structural; that fact does not propagate to the Ordering domain.

The construct's predicate extends to other architectural roles that admit the same analysis. Service discovery, queue-as-glue between bounded contexts, configuration-management roles, and distributed-tracing collectors are auditable under the two conditions of §3; the present experiment does not exercise them, and the substrate's predicted relation to each is left as a forward question. The construct admits this open extension by design — its diagnostic value is in identifying which categories in any given system are contingent, not in enumerating them exhaustively.

8. Conclusion

This paper has named a property that was previously implicit in distributed-systems practice: that the server-role is not necessarily structural to the problem domain but can arise from representational choices in the persistence model. The term *accidental category* designates this condition.

Two conditions have been stated that allow practitioners to distinguish accidental categories from structural requirements, and §3 has operationalized these conditions across canonical architectural roles. The instantiation of §4 demonstrates that, under a journaled-program substrate, the Ordering bounded context of `dotnet/eShop` continues to operate across three mutually bootstrapping nodes without any participant fulfilling the server-role. The demonstration does not argue uniqueness of the substrate; it establishes observability of the condition.

Under these conditions, the four functions traditionally attributed to the server-role — authoritative writes, coordination ownership, client mediation, and address provision — are observed to dissolve into properties of the substrate it-

self. What appears as "server behavior" is redistributed across peer rotation, mesh communication, disk-backed partition tolerance, and one-time bootstrap discharge.

The same observation extends to external infrastructure. Cloud services, microservices, and infrastructural ecosystems, when accessed from a journaled program, are no longer required habitats for the system to exist, but libraries invoked by the program when needed. The distinction is not eliminative but relational: from inhabitation to invocation.

The contribution of this paper is the formalization of a discrimination that can be applied independently of the substrate used in §4. The construct allows any production system to be examined role-by-role, asking whether each role is tied to the problem being solved or to the persistence model chosen.

The discrimination is per role-in-use-case, not per system. Identity authorities, payment networks, sensor integrations, and computationally intensive services remain structural under any substrate. The construct predicts no universal dissolution; it predicts examinability.

Paper 6 applied the construct at the layer level, identifying many infrastructural layers — caches, queues, locks, application servers, orchestration clusters — as compensations for the underlying persistence model. The present paper extends the same discrimination to the architectural role that made such layers necessary in the first place: the server. The two papers apply one construct at two levels of abstraction — first to layers, then to roles. From this point on, treating the server-role as a structural requirement is no longer an assumption but a choice that can be made explicit and evaluated.

Appendix A — Reproducibility

The artifact described in §4 will be released open-source with the publication of this paper. The references in this appendix to file paths, test method names, and orchestrator scripts are pointers into the forthcoming release. The artifact comprises the ported domain library, the host process, the orchestrator script, the test suite, and the supporting documentation in `notes/`.

A.1 Source and suite

The release will include the substrate framework, the ported `Ordering` domain library, the demo orchestrator, the test suite, and the supporting documentation referenced in this appendix. The exact branch, commit, and test-count metadata will be recorded with the release tag at the time of publication.

A.2 Demo orchestrator

The orchestrator script `docker/run-demo.sh` provisions three Docker containers, runs the analog bootstrap handshake against each, and exercises one cell of

the matrix per invocation. Three flags select the scenario:

- `bash docker/run-demo.sh` — cross-container inline (22 entries final).
- `bash docker/run-demo.sh --parametric` — cross-container parametric (7 entries final).
- `bash docker/run-demo.sh --rehydrate-demo` — captures the cross-container leave-and-rejoin scenario referenced in §7: stops one container, allows the cluster to advance, restarts the container, and observes its rehydration and catch-up.

Each invocation runs in approximately 60–90 seconds on Docker Desktop.

A.3 Tests cited

The empirical claims in the body are exercised by specific tests in the suite:

Body section	Property	Test method	Project
§4.4	F1–F5 bootstrap handshake under real cryptography	`EndToEnd_RealCryptography_TwoPeersOnboardingTests`	`UnitTestPaper7EShop/TwoNodeOnboardingTests`
§5 (single peer operates alone)	Force-promoted Stage operates without peers	`SingleStage_OperatesAlone_WhenPeersAbsent`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§4.5 (in-process, inline)	Three-stage Director rotation, inline regime	`ThreeStages_DirectorRotation_HappyPath`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§4.5 (in-process, parametric)	Three-stage Director rotation, parametric regime	`ThreeStages_DirectorRotation_HappyPath_Parametric`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§5 (rotation under cancellation)	Cancellation branch replicates across three stages	`CancellationBranch_ReplicatesAcrossThreeStages`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§4.5 (instrumentation)	DLL load + per-phase convergence + journal bytes snapshot	`Metrics_F1_Snapshot`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`
§4.6 / §5 (partition tolerance)	Leave-and-rejoin from disk, peer-to-peer catch-up	`OneStage_Offline_UntilAdvanced_RehydratesAndCatchesUp`	`UnitTestPaper7EShop/ThreeNodeOrderingTest`

Body section	Property	Test method	Project
§5 (rehydration without onboarding)	Rehydrated Stage restores identity from disk alone	`RehydratedStage_Restores`	`Unit`

A.4 Cryptographic stack

The cryptographic primitives underlying the F1–F5 handshake of §4.4 are documented in `Choreography/Usher/Crypto/*.cs`. The implementations use BouncyCastle 2.6.2:

- Ed25519 keypair generation, signing, verification: `Ed25519StageKeyGenerator.cs`, `Ed25519Signer.cs`, `Ed25519SignatureVerifier.cs`.
- Ed25519-to-X25519 derivation (Edwards-to-Montgomery, $u = (1 + y) / (1 - y) \bmod p$): `Ed25519ToX25519.cs`.
- Sealed-box AEAD using X25519 ECDH and ChaCha20-Poly1305, with the participating public keys bound as additional authenticated data: `SealedBoxPayloadSealer.cs`.
- Kestrel TLS listener and self-signed certificate generation with SAN: `HttpsTransportListener.cs`, `SelfSignedCert.cs`.
- TOFU certificate fingerprint pin via `SocketsHttpHandler.SslOptions.RemoteCertificateValidation` `HttpsClientFactory.cs`.

A.5 Share-link URI

The share-link emitted by the issuer takes the form `puppeteer-usher://v1/{base64url(json)}`. The JSON payload carries the transport address of the issuer endpoint and a SHA-256 fingerprint (`fp` field) of its TLS certificate. The encoder/decoder is defined by `IUsherShareLinkEncoder` in `Choreography/Usher/`. A representative share-link under load is 449 characters in length, within the alphanumeric capacity of a QR code at version 14 with error correction level M (approximately 535 characters).

A.6 Wire format

The HTTPS transport between containers carries a `ChannelPurpose` byte after the sender identifier in each frame. This wire format (v2) prevents Define and Invocation channels from collapsing into the same channel-dictionary key when Director rotation runs the parametric workload across multiple containers; the in-process regime is unaffected. The wire format is implemented in the transport handlers under `Choreography/Transport/Https/`.

A.7 Supporting reports

Two markdown reports in the `notes/` directory of the same branch document the experiment in detail:

- `notes/paper7_phase1_results.md` — the in-process row of the matrix (eleven sections plus three addenda).
- `notes/paper7_phase2_results.md` — the cross-container row, the hardening cycle that converged on the demo, the three-Docker mesh, the rotation protocol, and the parametric cell.

A.8 Recordings

Eleven cast recordings (asciinema format) and eleven GIF renders capture the four cells of the matrix of §4.5 and the rehydration property of §4.6, one recording per Docker container per cell. The recordings are published alongside this paper as supplementary material, hosted in the `paper7-assets/` directory of the `puppeteer-papers` repository.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Fowler, M. (2005). Event sourcing. Retrieved from https://martinfowler.com/eaaDev/EventSourcing.html

Harris-Braun, E., Luck, N., & Brock, A. (2018). *Holochain: Scalable agent-centric distributed computing* (Alpha 1 white paper). Holo. https://www.holochain.org/documents/holochain-white-paper-alpha.pdf

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Kleppmann, M., Frazee, P., Gold, J., Graber, J., Holmgren, D., Ivy, D., Johnson, J., Newbold, B., & Volpert, J. (2024). Bluesky and the AT Protocol: Usable decentralized social media. In *Proceedings of the ACM ConEXT-2024 Workshop on the Decentralization of the Internet*. ACM. https://doi.org/10.1145/3694809.3700740

Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Onward! 2019: Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (pp. 154–178). ACM. https://doi.org/10.1145/3359591.3359737

Lemmer-Webber, C., Tallon, J., Shepherd, E., Guy, A., & Prodromou, E. (2018). *ActivityPub* (W3C Recommendation, 23 January 2018). World Wide Web Consortium. https://www.w3.org/TR/activitypub/

Matrix.org Foundation. (n.d.). *Matrix specification* [Online document]. Retrieved from https://spec.matrix.org/

Ogden, M., McKelvey, K., & Madsen, M. B. (2017). *Dat: Distributed dataset synchronization and versioning* [White paper]. Code for Science. https://github.com/datprotocol/whitepaper

Rivera, A. (2026a). Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems. *Puppeteer Papers Series*, Paper 1. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/01-anti-porosity.md

Rivera, A. (2026b). Program–value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime. *Puppeteer Papers Series*, Paper 2. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/02-program-value-separability.md

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/03-reactions-and-partition.md

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/04-cross-actor-continuity.md

Rivera, A. (2026e). The journal as substrate: unifying deployment, replication,

backup, and offline operation in distributed systems. *Puppeteer Papers Series*, Paper 5. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/05-substrate-operations.md

Rivera, A. (2026f). Most infrastructure layers are symptoms of the persistence model: a construct for auditing production stacks. *Puppeteer Papers Series*, Paper 6. https://github.com/alvaroNCubo/puppeteer-papers/blob/main/06-infrastructural-symptom.md

Sambra, A. V., Mansour, E., Hawke, S., Zereba, M., Greco, N., Ghanem, A., Zagidulin, D., Aboulnaga, A., & Berners-Lee, T. (2016). *Solid: A platform for decentralized social applications based on linked data* (Technical Report). MIT CSAIL & Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

Tarr, D., Lavoie, E., Meyer, A., & Tschudin, C. (2019). Secure Scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN '19)* (pp. 1–11). ACM. https://doi.org/10.1145/3357150.3357396

Vernon, V. (2015). *Reactive messaging patterns with the actor model: Applications and integration in Scala and Akka.* Addison-Wesley.

Young, G. (2010). *CQRS documents.* Retrieved from https://cqrs.files.wordpress.com/2010/11/cqrs_document